
Que faire : délégation ou extension ?

Récapitulatif (classes abstraites)

- Classe abstraite : classe partiellement implémentée.
- Les classes abstraites sont une technique permettant de factoriser des parties communes à plusieurs classes.
- Extension : classe fabriquée à partir d'une autre classe par ajout de propriétés et de méthodes.
- Toute classe est une extension (sauf `Object`).
- L'extension redéfinit les méthodes, masque les propriétés du parent.
- La méthode exécutée est celle de la classe instanciée par `new`.
- syntaxe : `abstract`, `@Override`, `extends`, `super`, `protected`.

Classe abstraite vs interface

Classe abstraite :

- extension unique : chaque classe étend une seule autre classe.
- méthodes abstraites,
- possède des propriétés,
- visibilités variées (`protected`, `private`,...).

Interface :

- implémentation multiple : une classe implémente autant d'interfaces que nécessaire.
- méthodes par défaut,
- pas de propriétés (sauf constantes, implicitement `public static final`),
- tout est public.

Méthodes par défaut

Interface et méthodes par défaut

Une classe abstraite peut avoir des méthodes implémentées (non abstraite).

Une interface aussi peut avoir des méthodes implémentées : des **méthodes par défaut** (**default**).

- Une interface n'a pas de propriété (sauf constantes).
- Les méthodes par défaut peuvent appeler les méthodes non-implémentées de l'interface.
- Les méthodes par défaut sont signalées avec le mot-clé **default**.
- Une implémentation peut redéfinir les méthodes par défaut.

Méthode par défaut, exemple

```
public interface Monoid<E> {  
    E neutral();  
    E operator(E value1, E value2);  
  
    default E reduce(List<E> values) {  
        E accum = neutral();  
        for (E value : values) {  
            accum = operator(accum, value);  
        }  
        return accum;  
    }  
}
```

Redéfinition de méthode par défaut

Les méthodes par défaut peuvent être redéfinies.

```
public class ConjonctionMonoid implements Monoid<Boolean>
{
    public boolean neutral() { return true; }
    public boolean operator(boolean b1, boolean b2) {
        return b1 && b2;
    }

    public boolean reduce(List<Boolean> bools) {
        for (Boolean b : bools) {
            if (!b) return false;
        }
        return true;
    }
}
```

Mise en situation

Les deux prochains transparents montrent deux classes très similaires.

Principe DRY : *Don't Repeat Yourself*, éviter les duplications dans le programme.

Problématique : comment réorganiser le programme pour éviter toute duplication ?

Example (I)

```
public class SumList {  
    List<Integer> ints;  
  
    public SumList(List<Integer> ints) {  
        this.ints = ints;  
    }  
  
    public int eval() {  
        int sum = 0;  
        for (Integer i : ints) {  
            sum = sum + i;  
        }  
        return sum;  
    }  
}
```

Example (II)

```
public class ProductList {  
    List<Integer> ints;  
  
    public ProductList(List<Integer> ints) {  
        this.ints = ints;  
    }  
  
    public int eval() {  
        int product = 1;  
        for (Integer i : ints) {  
            product = product * i;  
        }  
        return product;  
    }  
}
```

Analyse préliminaire

- même propriété, les méthodes diffèrent dans l'implémentation uniquement.
- les méthodes n'ont pas tout-à-fait les mêmes instructions : impossible de généraliser en ajouter des propriétés à l'objet.

Deux solutions :

- par **délégation** (interfaces),
- par **extension** (classes abstraites).

Interfaces et délégation

Factorisation, étape 1

```
public class SumList {
    List<Integer> ints;

    public SumList(List<integer> ints) {
        this.ints = ints;
    }

    public int eval() {
        int value = neutral();
        for (Integer i : ints) {
            value = operator(value,i);
        }
        return value;
    }

    private int operator(int a, int b) { return a + b; }
    private int neutral() { return 0; }
}
```

Factorisation : délégation

```
public class SumList {
    List<Integer> ints;
    private IntMonoid monoid;

    public SumList(List<integer> ints, IntMonoid monoid) {
        this.ints = ints;
        this.monoid = monoid;
    }

    public int eval() {
        int value = monoid.neutral();
        for (Integer i : ints) {
            value = monoid.operator(value,i);
        }
        return value;
    }
}
```

Factorisation : délégation (usage)

```
public interface Monoid<T> {  
    T neutral();  
    T operator(T a, T b);  
}  
  
public class SumMonoid implements Monoid<Integer> {  
    public int neutral() { return 0; }  
    public int operator(int a, int b) { return a + b; }  
}  
  
SumList list = new SumList(ints, new SumMonoid());  
int sum = list.eval();
```

Délégation

Une *délégation* consiste pour un objet déléguant à faire faire un travail par un autre objet délégué, propriété du déléguant.

Intérêt :

- garder la classe simple en exportant une partie du travail dans une autre classe (chaque classe a un seul rôle),
- permettre de paramétrer la classe selon le délégué. C'est donc une technique de factorisation.
- on a ainsi une classe pour les parties communes, et une classe pour chaque comportement distinct.

Résolution de l'exemple

Dans notre exemple, la délégation porte sur la partie qui diffère entre les deux classes :

- la valeur initiale dans la sommation,
- l'opérateur arithmétique utilisé.

Le délégué est le monoïde possédant une valeur et un opérateur.

N'importe quel bloc d'instruction peut être délégué à un autre objet, c'est donc une solution flexible.

Classes abstraites et extensions

Deuxième solution : classe abstraite

Idée similaire : une classe avec la partie commune, des classes pour chaque comportement distinct.

Réalisation :

- faire une classe à *trous*, certaines méthodes restent sans implémentation (classe abstraite).
- faire des classes bouchant les trous : on implémente uniquement les méthodes qui ne sont pas implémentées par la classe abstraite (extension).
- la même classe à trou peut être complétée de multiples fois (plusieurs extensions)

Exemple de classe abstraite

```
public abstract class OperatorList {
    public abstract int neutral();
    public abstract int operator(int a, int b);

    protected List<Integer> ints;

    public OperatorList(List<Integer> ints) {
        this.ints = ints;
    }

    public int eval() {
        int value = neutral();
        for (int i : ints) {
            value = operator(value,i);
        }
        return value;
    }
}
```

Exemple d'extension

```
public class SumList extends OperatorList {
    public int neutral() { return 0; }
    public int operator(int a, int b) { return a + b; }

    public SumList(List<Integer> ints) {
        super(ints);
    }
}

public class ProductList extends OperatorList {
    public int neutral() { return 1; }
    public int operator(int a, int b) { return a * b; }

    public ProductList(List<Integer> ints) {
        super(ints);
    }
}
```

Déclaration, super, extension

Une classe abstraite est définie avec **abstract** :

```
public abstract class MyAbstractClass { ... }
```

Une extension est définie avec **extends** :

```
public class MyConcreteClass extends MyAbstractClass {  
    ...  
}
```

MyAbstractClass est alors la **super**classe de MyConcreteClass. MyConcreteClass est une *extension* (ou *sous-classe*) de MyAbstractClass.

Méthodes abstraites ou implémentées

```
public abstract class Parent {  
    protected int value = 0;  
  
    public void f() {  
        println("f");  
    }  
    public abstract void g();  
}
```

```
public class Child  
    extends Parent {  
    public void g() {  
        println("g");  
    }  
    public void h() {  
        println(this.value);  
    }  
}
```

- La classe Parent a une méthode implémentée f, une méthode abstraite g, une propriété value.
- L'extension Child a pour méthodes implémentées f, g, h et pour propriété value.
- L'extension Child pourrait redéfinir la méthode f si nécessaire.

Classes abstraites et propriétés

Une classe abstraite peut avoir des propriétés, commune à toutes les extensions.

Une propriété **privée** d'une classe abstraite **est inaccessible** depuis les extensions. Nouvelle visibilité **protected** :

- **private** : inaccessible dans les extensions ;
- **protected** : **accessible dans la classe et ses extensions uniquement** ;
- sans visibilité explicite (*package-private*) : accessible dans la classe et tout son package (mais pas dans les extensions hors du package) ;
- **public** : accessible partout.

protected peut aussi s'appliquer aux méthodes, constructeurs, ...

Classes abstraites et constructeurs

Une classe abstraite **ne peut pas** être instanciée.

Une classe abstraite **peut** néanmoins posséder un constructeur :

- pour factoriser une partie de la construction du nouvel objet, quelque soit l'extension.
- les extensions **doivent** avoir un constructeur, qui appelle le constructeur de la superclasse (mot-clé **super**). Par défaut le constructeur de **super** sans argument est appelé.

```
public SumList(List<Integer> ints) {  
    super(ints);  
}
```

Redéfinition de méthode et super

L'extension peut faire appel à une méthode de sa superclasse avec le mot-clé `super`. Utile si l'extension `redéfinit` une méthode concrète de la superclasse.

```
abstract class A {  
    public void foo() { System.out.println "A.foo"; }  
}  
class B extends A {  
    public void foo() {  
        super.foo();  
        System.out.println("B.foo");  
    }  
}  
B b = new B();  
b.foo(); // "A.foo", "B.foo"
```

Quelle méthode est exécutée ?

Rappel :

Typage

C'est le type déclaré de la variable qui détermine quelles méthodes **peuvent être appelées**.

Polymorphisme

C'est la classe d'instanciation de l'objet qui détermine **quelles instructions sont exécutées** lors d'un appel de méthode.

Mécanisme : les instructions sont choisies en suivant la table de méthodes de l'objet, dont la référence est stockée avec l'objet dans le tas, lors de son instanciation.

Extensions générales

Les classes non-abstraites peuvent aussi être étendues :

- hérite des méthodes et propriétés de la superclasse,
- possède des nouvelles fonctionnalités (méthodes et propriétés),
- redéfinit des méthodes de la superclasse avec d'autres comportements.

Limitations :

- en Java, on ne peut étendre qu'une seule classe,
- on peut déclarer une classe finale, ce qui interdit de l'étendre,
- les extensions peuvent être à l'origine de problèmes subtils.

La classe Object

En Java, toute classe étend la classe Object.

```
boolean equals(Object obj);  
int hashCode();  
String toString();  
...
```

```
Point point = new Point(3,2);  
System.out.println(point.toString());  
    // --> "test.Point@8c24e1"  
System.out.println(point);  
    // --> "test.Point@8c24e1"
```

Rappels sur les interfaces

Objets, classes, interfaces

- Les objets ont des propriétés et des comportements.
- Les classes regroupent des objets ayant :
 - des propriétés de même nature, mais pas de même valeur,
 - des comportements de même nature et de même valeur.
- les interfaces regroupent des objets ayant :
 - des comportements de même nature, mais différents.

Exemple des recyclables

Imaginons les classes : Papier, Canette, Bouteille, Carton.
Toutes ont des comportements différents, s'utilisent dans des contextes différents, possède une méthode `recycle`.

Nous voulons une classe des poubelles spécifiques aux déchets recyclables.

Consulter l'exemple `cours5.recyclableV1` et l'amélioration `cours5.recyclableV2`.

Interface

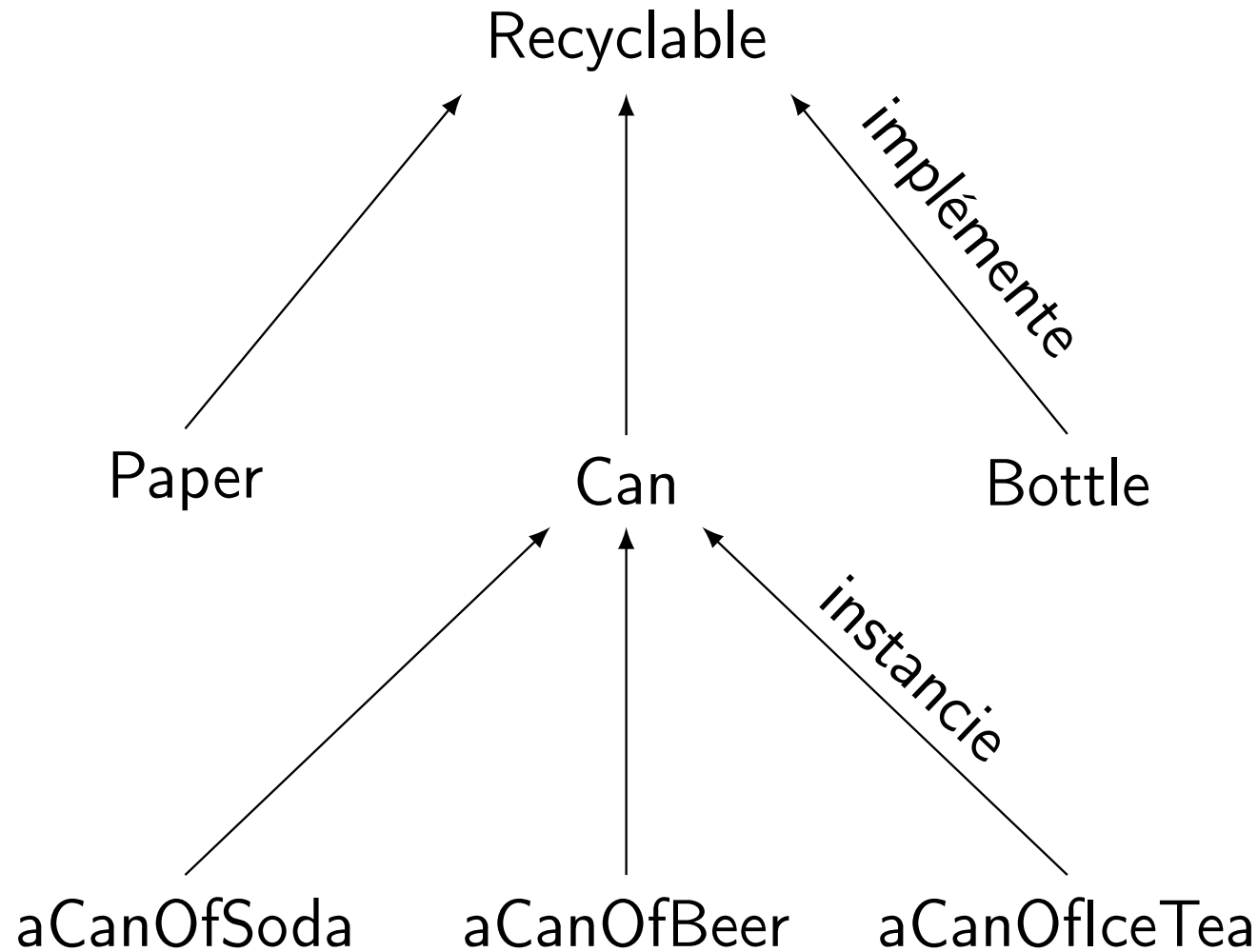
Une *interface* décrit un ensemble de fonctionnalités communes à plusieurs classes.

Pour nos objets, la fonctionnalité commune est la recyclabilité. *Recyclable* est une interface !

Une interface

Une *interface* est la description d'une liste de services, sans préciser comment le service est accompli (sans implémentation).

Un niveau d'abstraction supplémentaire



Implémentation d'une interface

Implémenter une interface

Une classe (ou un objet) *implémente* une interface si toute méthode déclarée dans l'interface est déclarée et programmée (implémentée) dans la classe.

- Pour implémenter une méthode d'une interface, la méthode de la classe doit avoir le même nom, les mêmes paramètres (de même types), le même type de retour.
- Une classe peut implémenter des méthodes en plus par rapport à l'interface.
- Une classe peut implémenter plusieurs interfaces différentes.

Interfaces et types

Types

Toute interface définit un *type*.

- N'importe quel paramètre, variable, propriété, peut être déclaré de ce type.
- Le type de retour d'une méthode peut être une interface.
- N'importe quel objet `obj` **instanciant** une classe `ClassName` **implémentant** une interface `InterfaceName` a les **deux types** `ClassName` **et** `InterfaceName`.

Exemples d'interface comme type

```
Paper item = new Paper(); // OK
Recyclable item = new Paper(); // OK
Paper item = new Recyclable(); // NO
Recyclable item = new Recyclable(); // NO
Paper item = new Can(); // NO
```

- Paper implémente Recyclable : ligne 2 OK.
- Recyclable est une interface, une interface **n'est pas instanciable** : lignes 3 et 4 incorrectes.
- Paper et Can implémentent Recyclable, mais sont deux classes distinctes qu'on ne peut confondre : ligne 5 incorrecte.

Types et méthodes

Typage

C'est le type déclaré de la variable qui détermine comment la variable peut être utilisée, en particulier quelles méthodes peuvent être appelées.

Les **contraintes de typage nous aide**, en rendant impossible l'appel d'une méthode d'un objet n'ayant pas cette méthode.

Polymorphisme

C'est l'objet (sa classe) qui détermine quelles instructions sont exécutées, lors d'un appel de méthode (le type n'intervient pas dans le choix de la méthode).